== vs ===

== converts types, === does not. Plus typeof quirks

typeof always returns a string 7 primitives, a few surprises

42 number	"hi" string	true boolean
undefined undefined	10n bigint	Symbol() symbol
null object	[] object	NaN number
undeclared undefined	() => {} function	class A {} function

Lime = the traps. **let/const** before their line throw ReferenceError.

1 == converts types

```
1 == "1"           // true
"" == 0            // true
[] == false        // true
10n == 10          // true
null == undefined  // true
null == 0          // false
```

Many rules. **null** equals only **undefined**.

2 === compares as is

```
1 === "1"          // false
{} === {}          // false
NaN === NaN        // false
+0 === -0          // true
Object.is(NaN,NaN) // true
Object.is(0,-0)    // false
```

Objects compare by **reference**.

TRAP What does this log?

```
console.log(typeof NaN, NaN === NaN);
```

number false

NaN is a number that never equals itself.

KEY TAKEAWAY

- `===` Your default
- `== null` Catches null and undefined
- `typeof` Not for null or arrays

Follow-up questions to expect

- **Why is `typeof null` "object"?** → A type-tag bug in the first engine, kept.
- **`NaN` : how do you test for it?** → Number.isNaN(x). isNaN("abc") is true.
- **When is `==` acceptable?** → x == null: matches null and undefined.
- **`Object.is` vs `===` : what differs?** → NaN equals NaN. +0 and -0 differ.